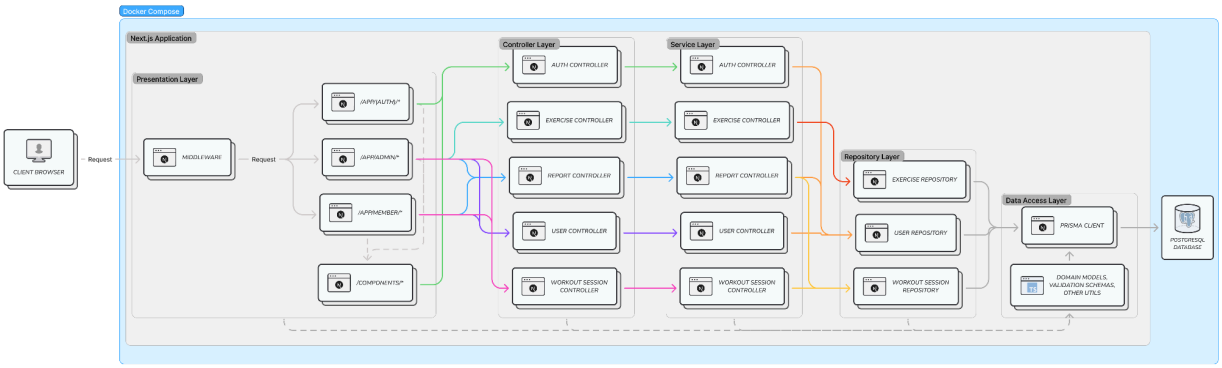
Gym Tracker Pro

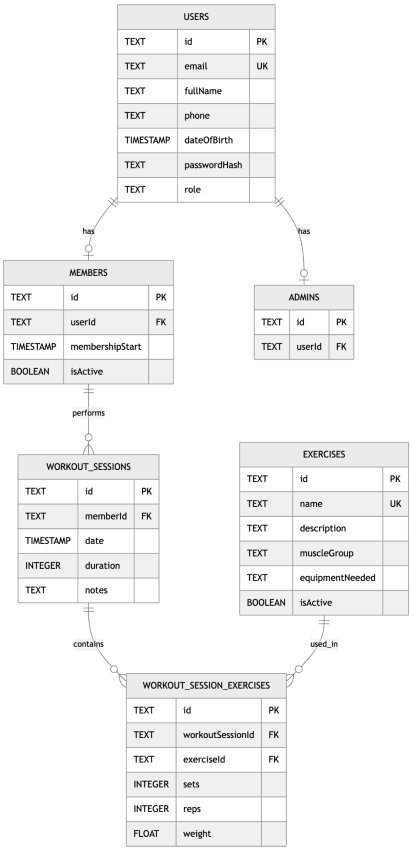
Design Specification

Architectural Overview



Entity & Data Model

Entity Relationship Diagram



Data Integrity Invariants

- A User cannot exist without a unique email, a role, and all personal fields (fullName, phone, dateOfBirth, passwordHash).
- A Member cannot exist without a valid userId referencing an existing User.
- An Admin cannot exist without a valid userId referencing an existing User.
- A User can have at most one Member profile and at most one Admin profile.
- A WorkoutSession cannot exist without a valid memberId referencing an existing Member.
- A WorkoutSessionExercise cannot exist without a valid workoutSessionId and a valid exerciseId.
- An Exercise cannot exist without a unique name, a muscleGroup, and an equipmentNeeded value.
- Deleting a User cascades to their Member or Admin profile.
- Deleting a Member cascades to all their WorkoutSession rows.
- Deleting a WorkoutSession cascades to all its WorkoutSessionExercise rows.
- Deleting an Exercise is blocked (RESTRICT) if any WorkoutSessionExercise references it - historical session data is preserved.
- Archiving an Exercise (isActive = false) does not affect existing WorkoutSessionExercise rows - references are retained for historical accuracy.
- Deactivating a Member (isActive = false) does not delete their WorkoutSession rows - session history is preserved.
- A WorkoutSessionExercise must always carry explicit values for sets, reps, and weight - no defaults exist for these fields.

Module Responsibilities

Domain Layer

File	Responsibility
user.ts	Re-exports **User**, **Member**, **Admin** from Prisma. Defines composite types: **UserWithProfile**, **MemberWithUser**, **AdminWithUser**, **MemberWithUserAndTempPassword**. Defines list-option types: **MemberListOptions**, **AdminListOptions**.
user-schema.ts	Exports Zod validation schemas: **createMemberSchema**, **createMemberWithTempPasswordSchema**, **createAdminSchema**, **updateMemberSchema**, **updateAdminSchema**, **loginUserSchema**.
exercise.ts	Re-exports **Exercise** from Prisma. Re-exports enums **MuscleGroup** (CHEST, SHOULDERS, ARMS, BACK, CORE, LEGS) and **Equipment** (CABLE, DUMBBELL, BARBELL, MACHINE). Defines **ExerciseListOptions** for filtering and pagination.
exercise-schema.ts	Exports Zod validation schemas: **createExerciseSchema**, **updateExerciseSchema**.

workout-session.ts	Re-exports **WorkoutSession** and **WorkoutSessionExercise** from Prisma. Defines **WorkoutSessionWithExercises** (session with nested exercises and their catalogue entries). Defines **WorkoutSessionListOptions** for filtering and pagination.
workout-session-schema.ts	Exports Zod validation schemas: **createWorkoutSessionSchema**, **updateWorkoutSessionSchema**, **workoutSessionExercisesSchema**, **workoutSessionExercisesUpdateSchema**.
session.ts	Exports **SessionData** type: shape of the authenticated user stored in the server-side session cookie. Fields: **userId**, **email**, **fullName**, **role**, optional **memberId** and **adminId**.
report.ts	Exports report aggregate types: **Report** (memberId, memberName, startDate, endDate, totalSessions, totalVolume, averageSessionDuration, exerciseBreakdown[], sessionDetails[]), **ExerciseStats** (exerciseId, exerciseName, muscleGroup, totalSets, totalReps, totalVolume, sessionCount), **SessionDetail** (sessionId, date, durationMinutes, notes?, exercises[], totalVolume), **SessionExerciseDetail** (exerciseId, exerciseName, sets, reps, weight, volume).
errors.ts	Exports the custom error hierarchy. Base class **AppError** extends **Error** with normalised **name** assignment. Subclasses: **NotFoundError**, **ConflictError**, **AuthorizationError**, **WorkoutSessionRequiresExercisesError**, **TransactionError**. Thrown by repositories and services, caught exclusively by controllers.
pagination.ts	Exports shared utility types. **PageResult<T>** wraps a paginated query result: **{ items: T[], total: number }**.
action-result.ts	Exports **ActionResult<T>**, the discriminated union returned by every Server Action: **{ success: true, data: T }** | **{ success: false, message: string, errors?: Record<string, string[]> }**.

Repository Layer

File	Responsibility
user-repository.ts	Prisma-backed implementation of **UserRepositoryInterface**. Consolidates all **Member** and **Admin** data access.

	• **createMember(data: CreateMemberInput): Promise<MemberWithUser>** - Hash password (bcrypt, 12 rounds), throw **ConflictError** if email taken, create **User** + **Member** atomically. • **createAdmin(data: CreateAdminInput): Promise<AdminWithUser>** - Hash password, throw **ConflictError** if email taken, create **User** + **Admin** atomically. • **findById(id): Promise<UserWithProfile>** - Get user with both profiles included. Throw **NotFoundError** if absent. • **findMemberById(memberId): Promise<MemberWithUser>** - Throw **NotFoundError** if absent. • **findAdminById(adminId): Promise<AdminWithUser>** - Throw **NotFoundError** if absent. • **findByEmail(email): Promise<UserWithProfile | null>** - Used by **AuthService** during login. Returns null if not found. • **findMembers(options): Promise<PageResult<MemberWithUser>>** - Paginated list, searchable by fullName or email (case-insensitive, SQL LIKE safe), sorted by fullName ascending. • **findAdmins(options): Promise<PageResult<AdminWithUser>>** - Same pattern as findMembers. • **updateMember(memberId, data): Promise<MemberWithUser>** - Atomic update of **Member** and parent **User** fields. Throw **ConflictError** on duplicate email. • **setMemberActive(memberId, isActive): Promise<MemberWithUser>** - Suspend or reactivate a member. • **updateAdmin(adminId, data): Promise<AdminWithUser>** - Same pattern as updateMember. • **delete(id): Promise<void>** - Resolves whether id is a member or admin, then deletes parent **User** (cascades to profile). Throw **NotFoundError** if absent.
exercise-repository.ts	Prisma-backed implementation of **ExerciseRepositoryInterface**. • **create(data: CreateExerciseInput): Promise<Exercise>** - Throw **ConflictError** if name already in use.

	• **findById(id): Promise<Exercise>** - Throw **NotFoundError** if absent. • **findAll(options): Promise<PageResult<Exercise>>** - Paginated list. Supports search by name (LIKE safe), filter by muscleGroup, and includeInactive flag (default false). Sorted by name ascending. • **update(id, data): Promise<Exercise>** - Throw **NotFoundError** or **ConflictError** on duplicate name. • **setActive(id, isActive): Promise<Exercise>** - Archive or unarchive. Throw **NotFoundError** if absent. • **delete(id): Promise<void>** - Throw **NotFoundError** if absent. Throw **ConflictError** if the exercise is referenced by any **WorkoutSessionExercise** row (referential RESTRICT).
workout-session-repository.ts	Prisma-backed implementation of **WorkoutSessionRepositoryInterface**. Manages both sessions and their exercises. • **create(data, exercises): Promise<WorkoutSessionWithExercises>** - Throw **WorkoutSessionRequiresExercisesError** if exercises list is empty. Verify member exists (**NotFoundError**). Create session + all exercises atomically. Wrap in **TransactionError** on DB failure. • **findById(id): Promise<WorkoutSessionWithExercises>** - Include exercises and their catalogue entries. Throw **NotFoundError** if absent. • **findAll(options): Promise<PageResult<WorkoutSessionWithExercises>>** - Supports memberId, startDate/endDate range, and pagination. Paginated results sorted descending by date. • **update(id, data): Promise<WorkoutSession>** - Update metadata only (date, duration, notes). Throw **NotFoundError** if absent. • **updateWithExercises(id, data, exercises): Promise<WorkoutSessionWithExercises>** - Atomic full replacement of session metadata and exercises in a single transaction. Deletes removed exercises, updates existing (by id), inserts new. Throw **WorkoutSessionRequiresExercisesError** if result would have zero exercises. Wrap in **TransactionError** on DB failure.

	● **delete(id): Promise<void>** - Throw **NotFoundError** if absent. Cascade to exercises handled by DB.

Service Layer

File	Responsibility
auth-service.ts	Implements **AuthServiceInterface**. Handles credential verification. ● **login(data: LoginUserInput): Promise<SessionData>** - Look up user by email via **userRepository.findByEmail()**. Compare password with bcrypt. Throw **AuthorizationError** on invalid credentials (same message for both cases to prevent enumeration). Return **SessionData** on success.
user-service.ts	Implements **UserServiceInterface**. Manages both **Member** and **Admin** lifecycle. ● **createMember(data): Promise<MemberWithUser>** - Delegate to **userRepository.createMember()**. ● **createMemberWithTempPassword(data): Promise<MemberWithUserAndTempPassword>** - Generate a cryptographically random 16-character password (guaranteed uppercase, digit, special char via **crypto.getRandomValues**). Call **createMember()** with the temp password. Return member + plaintext temp password. ● **createAdmin(data): Promise<AdminWithUser>** - Delegate to **userRepository.createAdmin()**. ● **getMember(memberId): Promise<MemberWithUser>** / **getAdmin(adminId): Promise<AdminWithUser>** - Delegate to repository. ● **listMembers(options?): Promise<PageResult<MemberWithUser>>** / **listAdmins(options?): Promise<PageResult<AdminWithUser>>** - Delegate to repository. ● **updateMember(memberId, data): Promise<MemberWithUser>** / **updateAdmin(adminId, data): Promise<AdminWithUser>** - Delegate to repository. ● **suspendMember(memberId): Promise<MemberWithUser>** - Call **userRepository.setMemberActive(memberId, false)**.

	• **activateMember(memberId): Promise<MemberWithUser>** - Call **userRepository.setMemberActive(memberId, true)**. • **deleteMember(memberId): Promise<void>** / **deleteAdmin(adminId): Promise<void>** - Delegate to **userRepository.delete()**.
exercise-service.ts	Implements **ExerciseServiceInterface**. Manages the exercise catalogue. • **createExercise(data): Promise<Exercise>** - Delegate to repository. • **getExercise(id): Promise<Exercise>** - Delegate to repository. • **listExercises(options?): Promise<PageResult<Exercise>>** - Delegate to repository (supports search, muscleGroup filter, includeInactive, pagination). • **updateExercise(id, data): Promise<Exercise>** - Delegate to repository. • **archiveExercise(id): Promise<Exercise>** - Call **exerciseRepository.setActive(id, false)**. • **unarchiveExercise(id): Promise<Exercise>** - Call **exerciseRepository.setActive(id, true)**. • **deleteExercise(id): Promise<void>** - Delegate to repository (blocked if exercise is referenced).
workout-session-service.ts	Implements **WorkoutSessionServiceInterface**. Thin delegation layer over the repository. • **createWorkoutSession(data, exercises): Promise<WorkoutSessionWithExercises>** - Delegate to repository. • **getWorkoutSession(id): Promise<WorkoutSessionWithExercises>** - Delegate to repository. • **listMemberWorkoutSessions(memberId, options?): Promise<PageResult<WorkoutSessionWithExercises>>** - Delegate to repository with memberId merged into options. • **updateWorkoutSession(id, data): Promise<WorkoutSession>** - Update metadata only.

	• **updateWorkoutSessionWithExercises(id, data, exercises): Promise<WorkoutSessionWithExercises>** - Atomic full replacement of metadata and exercises. • **deleteWorkoutSession(id): Promise<void>** - Delegate to repository.
report-service.ts	Implements **ReportServiceInterface**. Depends on both **WorkoutSessionRepositoryInterface** and **UserRepositoryInterface**. • **getMemberProgressReport(memberId, startDate, endDate): Promise<Report>** - Fetch member (throw **NotFoundError** if absent). Fetch all sessions in range. Aggregate in-memory using Map/reduce: compute per-exercise stats (totalSets, totalReps, totalVolume, sessionCount via Set deduplication), per-session totals, overall totalVolume, averageSessionDuration. Return sorted exerciseBreakdown (descending by totalVolume).

Controller Layer

File	Responsibility
auth-controller.ts	Exports the following Server Actions: • **login(data: LoginUserInput): Promise<ActionResult<SessionData>>** - Validate via **loginUserSchema**. Call **authService.login()**. Create an iron-session cookie (httpOnly, secure in production). Return **ActionResult<SessionData>** or field errors. Catch **AuthorizationError** -> form banner. • **logout(): Promise<ActionResult<void>>** - Destroy session. Return **{ success: true }**.
user-controller.ts	Exports the following Server Actions for both member and admin management: • **createMember(data) / createMemberWithTempPassword(data) / createAdmin(data)** - Validate via respective Zod schema. Delegate to **userService**. Catch **ConflictError** (duplicate email) -> form banner. • **getMember(memberId) / getAdmin(adminId)** - Delegate to service. Catch **NotFoundError**.

	• **listMembers(options?) / listAdmins(options?)** - Return paginated **PageResult**. • **updateMember(memberId, data) / updateAdmin(adminId, data)** - Validate via Zod. Delegate to service. Catch **NotFoundError**, **ConflictError**. • **suspendMember(memberId) / activateMember(memberId)** - Delegate to service. Catch **NotFoundError**. • **deleteMember(memberId) / deleteAdmin(adminId)** - Delegate to service. Return **{ success: true }** or error.
exercise-controller.ts	Exports the following Server Actions: • **createExercise(data): Promise<ActionResult<Exercise>>** - Validate via **createExerciseSchema**. Catch **ConflictError** (duplicate name) -> form banner. • **getExercise(id) / listExercises(options?)** - Delegate to service. **listExercises** passes options through (search, muscleGroup, includeInactive, pagination). • **updateExercise(id, data)** - Validate via **updateExerciseSchema**. Catch **NotFoundError**, **ConflictError**. • **archiveExercise(id) / unarchiveExercise(id)** - Delegate to service. Catch **NotFoundError**. • **deleteExercise(id)** - Delegate to service. Catch **NotFoundError**, **ConflictError** (exercise in use).
workout-session-controller.ts	Exports the following Server Actions: • **createWorkoutSession(data, exercises): Promise<ActionResult<WorkoutSessionWithExercises>>** - Validate session via **createWorkoutSessionSchema** and exercises via **workoutSessionExercisesSchema**. Delegate to service. Catch **WorkoutSessionRequiresExercisesError**, **NotFoundError**. • **getWorkoutSession(id) / listMemberWorkoutSessions(memberId, options?)** - Delegate to service. • **updateWorkoutSession(id, data)** - Validate via **updateWorkoutSessionSchema**. Update metadata only. • **updateWorkoutSessionWithExercises(id, data, exercises)** - Validate both session and exercises via Zod.

	Atomic full replacement. Catch **WorkoutSessionRequiresExercisesError**, **TransactionError**. ● **deleteWorkoutSession(id)** - Delegate to service. Catch **NotFoundError**.
report-controller.ts	Exports the following Server Actions: ● **getMemberProgressReport(memberId, startDate, endDate): Promise<ActionResult<Report>>** - Validate all three fields via **memberProgressReportSchema**. Delegate to **reportService.getMemberProgressReport()**. Catch **NotFoundError** (member absent) -> error banner. Catch unexpected -> generic message.

Note: All controllers return a typed ActionResult.

Presentation Layer - Admin Routes

Route	Component	Responsibility
/admin/dashboard	app/admin/dashboard/page.tsx	Admin overview. Fetches session server-side and total member count via listMembers({page:1, pageSize:1}). Renders a StatCard with the total member count and a personalised welcome header.
/admin/members	app/admin/members/page.tsx	Paginated, searchable member list. Accepts searchParams: search and page. Calls listMembers({search, page, pageSize:10}). Renders MembersTable component. Links to detail, edit, and suspend/activate actions.
/admin/members/new	app/admin/members/new/page.tsx + create-member-form.tsx	Member creation form. Renders CreateMemberForm (client component). Form action: createMemberWithTempPassword() server action. On success: display the one-time temporary password to the admin. On error: show inline field errors and form banner.
/admin/members/[id]	app/admin/members/[id]/page.tsx	Member detail page. Fetches via getMember(). Renders MemberDetailInformation (fullName, email, phone, dateOfBirth, membershipStart, isActive) and MemberRecentWorkoutSessions. Links to edit, report, suspend/activate.
/admin/members/[id]/edit	app/admin/members/[id]/edit/page.ts	Member edit form. Fetches current data via getMember(). Pre-fills EditMemberForm. Form

	x + edit-member-form.tsx	action: updateMember() server action. Catch ConflictError for duplicate email. On success: redirect to member detail.
/admin/members/[id]/report	app/admin/members/[id]/report/page.tsx	Progress report for a specific member. Inputs: startDate, dateTo ISO date pickers. Calls getMemberProgressReport(memberId, startDate, endDate). Renders report stats (totalSessions, totalVolume, averageSessionDuration), exercise breakdown table, and chronological session details.
/admin/exercises	app/admin/exercises/page.tsx	Exercise catalogue. Paginated, filterable table. Accepts searchParams: search, page, muscleGroup. Calls listExercises({includeInactive:true, ...options}). Renders ExercisesTable with archive/unarchive toggle, edit link, and delete button per row.
/admin/exercises/new	app/admin/exercises/new/page.tsx + create-exercise-form.tsx	Exercise creation form. Fields: name, description (optional), muscleGroup dropdown, equipmentNeeded dropdown. Form action: createExercise(). On success: redirect to /admin/exercises. Catch ConflictError for duplicate name.
/admin/exercises/[id]/edit	app/admin/exercises/[id]/edit/page.tsx + edit-exercise-form.tsx	Exercise edit form. Fetches current data via getExercise(). Pre-fills EditExerciseForm. Form action: updateExercise(). On success: redirect to /admin/exercises.
/admin/workout-sessions	app/admin/workout-sessions/page.tsx	Paginated workout session list across all members. Calls listMemberWorkoutSessions() with optional memberId filter. Renders sessions table with date, duration, exercise count.
/admin/workout-sessions/new	app/admin/workout-sessions/new/page.tsx + create-workout-session-form.tsx	Log a new workout session. Fields: member dropdown (from listMembers()), date, duration (minutes), optional notes. Dynamic exercise rows: exercise dropdown (active only via listExercises({includeInactive:false})), sets, reps, weight. Min 1 exercise enforced client and server-side. Form action: createWorkoutSession(). Atomic create via single DB transaction.
/admin/workout-sessions/[id]/edit	app/admin/workout-sessions/[id]/edit/page.tsx + edit-workout-sessi	Edit session and exercises atomically. Fetches current session via getWorkoutSession(). Renders EditWorkoutSessionForm with pre-filled metadata and existing exercises. Supports

	on-form.tsx	adding new rows and removing existing ones. Form action: updateWorkoutSessionWithExercises() - full atomic replacement. Min 1 exercise enforced.

Note: All controllers are Next.js Server Actions tagged with 'use server'. All return a typed ActionResult<T>.

Route	Component	Responsibility
/member/dashboard	app/member/dashboard/page.tsx	Welcome dashboard. Reads session server-side for fullName and memberId. Renders DashboardRecentSessions component showing the member's most recent sessions. Links to workout sessions list, progress report, and profile.
/member/profile	app/member/profile/page.tsx	Member profile page with three sub-components: ProfilePersonalInformation (fullName, email, phone, dateOfBirth - read from session), ProfileMembership (membershipStart, isActive status - fetched via getMember()), ProfileChangePasswordForm (client component, calls updateMember() with new password).
/member/workout-sessions	app/member/workout-sessions/page.tsx	Paginated list of own sessions (read-only). Accepts searchParams: page. Calls listMemberWorkoutSessions(session.memberId, {page, pageSize:10}). Renders WorkoutSessionsTable sorted descending by date. Links to session detail.
/member/workout-sessions/[id]	app/member/workout-sessions/[id]/page.tsx	Session detail (read-only). Calls getWorkoutSession(id). Renders WorkoutSessionStats (date, duration, notes) and WorkoutSessionExercisesTable (exerciseName, sets, reps, weight per row).
/member/report	app/member/report/page.tsx	Progress report scoped to own data. Inputs: startDate, endDate ISO date pickers. Calls getMemberProgressReport(session.memberId, startDate, endDate). The controller and service enforce that a member can only access their own report. Renders the same report components as the admin report page.

Shared Components

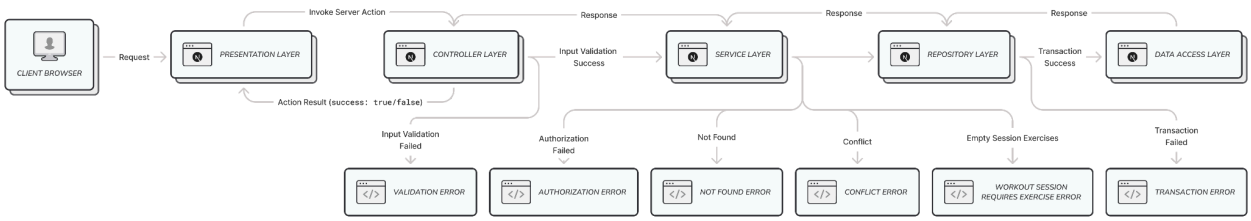
Component	Responsibility

components/auth/logout-button.tsx	Client component. Calls logout() server action on click and redirects to /login.
components/layout/page-header.tsx	Server component. Renders a page title, optional description, and optional action slot (e.g. Add Member button).
components/layout/pagination.tsx	Client component. Renders page navigation controls. Props: page, pageSize, total, baseUrl, searchParams. Uses Next.js router for navigation.
components/layout/sidebar-nav.tsx	Navigation sidebar. Renders role-specific links passed as props.
components/data/search-input.tsx	Client component. Debounced search input that updates the URL search param on change.
components/data/stat-card.tsx	Server component. Renders a labelled metric card with title, value, and description.
components/report/report-stats.tsx	Renders the report summary section: totalSessions, totalVolume, averageSessionDuration.
components/report/report-exercise-breakdown.tsx	Renders the per-exercise breakdown table: exerciseName, muscleGroup, totalSets, totalReps, totalVolume, sessionCount.
components/report/report-workout-session-details.tsx	Renders the chronological session table with nested exercise rows: date, duration, notes, and per-exercise sets/reps/weight/volume.
components/report/report-filter.tsx	Client component. Date range pickers (startDate, endDate). On apply, calls getMemberProgressReport() and triggers a re-render.
components/ui/	shadcn/ui primitives: Button, Input, Select, Badge, Table, Textarea, Alert, Avatar, Card, Dialog, DropdownMenu, Label, Separator. All styled with Tailwind CSS.

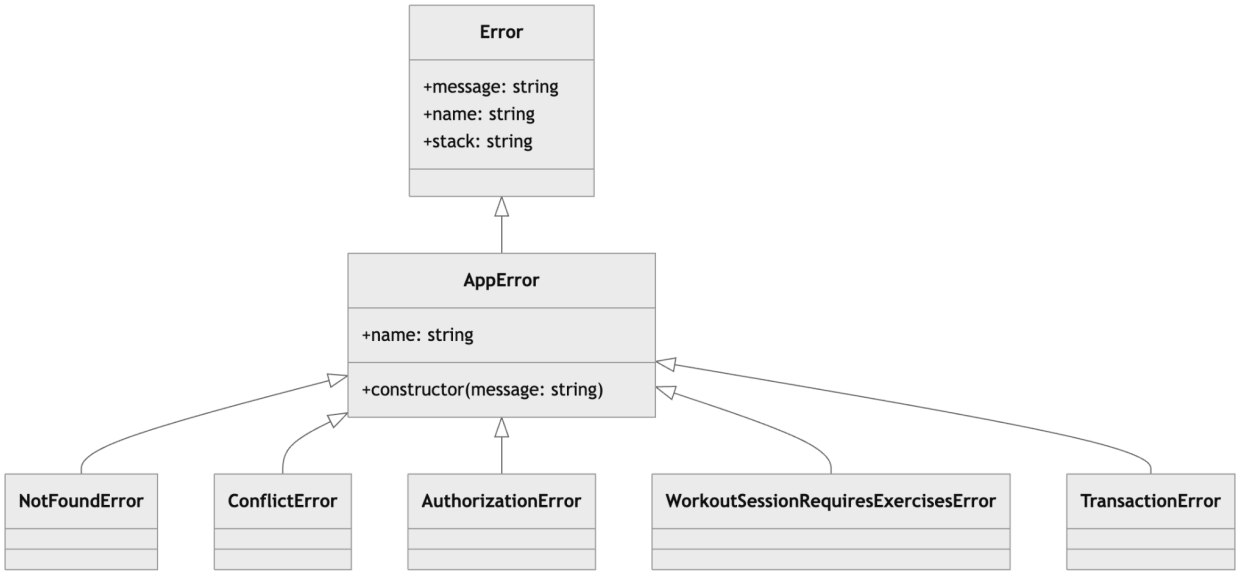
Error Handling Strategy

All errors are categorised into three tiers. No stack trace or internal detail is ever sent to the client.

Error Flow



Custom Error Hierarchy



Error Tier Summary

Tier	Source	Caught By	View	Logging
Validation	Zod schema failure in controller	Controller (Server Action)	Inline field errors per field (errors map)	-
Business Rule	Service/repository throws typed AppError subclass (NotFoundError, ConflictError, WorkoutSessionRequiresExercisesError, AuthorizationError)	Controller catches instanceof AppError	Form-level error banner with error.message	-
Infrastructure	Unexpected database or network error (non-AppError)	Controller catch-all	Generic "An unexpected error occurred" message	console.error (server-side only)

Design Patterns Applied

Pattern	Application	Description
Repository	/lib/repository/	Isolates all database interaction. Services, controllers, and the view have no ORM knowledge. Each repository

		implements a typed interface, enabling substitution without touching upstream layers.
Service Layer (Facade)	/lib/service/	Encapsulates business logic and multi-step operations. Controllers stay thin (validate -> call -> return). Services depend only on repository interfaces, not concrete implementations.
Singleton	/lib/database.ts, each repository and service	Single shared PrismaClient instance avoids connection pool exhaustion. Each service and repository exposes a static getInstance() factory that lazily creates and caches the single instance.
Dependency Injection (Composition Root)	/lib/di.ts	All singletons are wired together in one place. di.ts instantiates repositories with the shared Prisma client, then instantiates services with their required repositories. No service constructs its own dependencies.
Interface-based Programming	/lib/repository/*-interface.ts, /lib/service/*-interface.ts	Every repository and service is defined by a TypeScript interface. Concrete classes implement these interfaces. Controllers and services depend on the interface, not the implementation - supporting future substitution (e.g. swapping Prisma for another ORM, or mocking in tests).
Command	/lib/controller/	Each Server Action is a discrete named command (e.g. createWorkoutSession, suspendMember), independently callable and independently testable.
Strategy	/lib/service/report-service.ts	Report aggregation is fully encapsulated within the service using an in-memory Map/reduce strategy. The computation can be replaced or extended (e.g. push to a dedicated analytics store) without touching any other layer.

Accommodating Future Changes

Possible change	How the architecture handles it
Export report as PDF	Add a pdf-report-adapter.ts that accepts a Report object and renders it to PDF. No changes to report-service.ts or any controller.
Add charts to the report	Report already contains per-session, per-exercise arrays with all required data points. Only the report-* components need updating.
Swap PostgreSQL for another database	Change the datasource provider in schema.prisma and run prisma migrate. All other layers are insulated by the repository interfaces.
Add REST API endpoints	Add app/api/ route handlers that call the same controller functions or directly call services. No service or repository changes required.

Multi-gym / multi-tenant	Add a gymId FK to Member, Exercise, and WorkoutSession. Enforce the FK in all repository findAll() and create() calls. The repository layer is the only touch point.
Role expansion (e.g. Trainer role)	Add a new role to the Role enum in schema.prisma and a corresponding profile model. UserRepository.create*() handles the profile creation atomically. The session cookie already carries a role and profile id.
Email notifications (e.g. membership expiry)	Add a notification-service.ts that depends on UserRepositoryInterface. Wire it in di.ts. No existing service or controller needs to change.